Context Engineering Tools & Patterns

Cheatsheet · Hands-On Context Engineering — O'Reilly Live Training · Lucas Soares

> **WHAT IT IS**
> Context engineering is the discipline of designing the architecture that feeds an LLM the right information at the right time. It transforms high-entropy human intentions into low-entropy representations machines can process. Unlike humans, machines cannot "fill in the gaps."

The Four Context Operations (W / S / C / I)

Every context engineering strategy maps to one of these four fundamental operations.

- **WRITE** — Create context that doesn't exist yet. Plans, instructions, scratchpads, learned knowledge, memories. Agent writes files for future reference.
- **SELECT** — Choose the most relevant context from available sources. Semantic search, structural search (grep/glob), adaptive retrieval, dynamic tool loading.
- **COMPRESS** — Reduce size while preserving essential information. Summarization, pruning, offloading to disk, spawning sub-agents for token-heavy operations.
- **ISOLATE** — Separate context types to prevent interference. Sub-agent isolation, dynamic tool loading, working-memory scratchpads, state objects.

The Four Context Failure Modes

Drew Breunig's taxonomy — each mode has distinct symptoms and specific fix levers.

- **POISONING** — Hallucination enters context and compounds. Agent treats its own prior output as ground truth. *Signal:* bad fact reappears across steps. *Fix:* Isolate + Select
- **DISTRACTION** — Too much history causes the agent to repeat past behaviour instead of reasoning fresh. *Signal:* agent repeats prior actions. *Fix:* Compress + Select
- **CONFUSION** — Irrelevant tools or documents crowd the context, leading to wrong tool selection. *Signal:* irrelevant tools/docs selected. *Fix:* Select + Isolate
- **CLASH** — Contradictory information creates conflicting assumptions and unreliable output. *Signal:* flip-flopping between approaches. *Fix:* Isolate + Select

Quick Reference: Failures to Fixes

FAILURE	PRIMARY FIX	SUPPORTING FIX	MOVE
Poisoning	Isolate + Select	Write (claims + sources)	Prune hallucinated turns, add validation instructions
Distraction	Compress + Select	Write (persistent plan)	Summarize stale history, select a smaller working set
Confusion	Select + Isolate	Tool loadout (dynamic)	Load only relevant tools per step, isolate nice-to-have context
Clash	Isolate + Select	Compress (resolved summary)	Quarantine tasks into separate contexts, set precedence rules

The Five-Step Debugging Loop

- **1. Inspect trace** — Walk through messages, tool calls, and outputs turn by turn.
- **2. Tag failure** — Which mode? Poisoning / Distraction / Confusion / Clash.
- **3. Pick a lever** — Use the Write / Select / Compress / Isolate framework.
- **4. Re-run & evaluate** — Track token usage per step *and* measure outcome quality.
- **5. Iterate** — Context engineering is iterative, not one-shot.

> **DON'T OPTIMIZE THE WRONG THING**
> Don't just optimize for fewer tokens — measure outcome quality. A fix that reduces tokens but degrades results is worse than no fix at all.

The Context Budget Problem

Every agent must make critical decisions about its finite context budget.

- **Keep active** — What stays in the context window right now?
- **Store externally** — What goes to disk or a database for later?
- **Compress** — What can be summarized to save space?
- **Reserve** — How much capacity is left for reasoning?

> **BIGGER WINDOWS ARE NOT BETTER**
> Models support 100K, 200K, even 1M+ tokens — but performance degrades well before the limit. Gemini 2.5 Pro showed decreased performance beyond 100K tokens; Llama 3.1 405B hit its distraction ceiling at ~32K. **2026 update — the ceiling moved, but it did not disappear:** on MRCR v2 (8-needle) at 1M tokens, reported scores still split hard — Claude Opus 4.6 76%, GPT-5.4 36.6%, Gemini 3 Pro 24.5%. Same nominal window, very different *usable* window. (Vendor-reported; MRCR measures retrieval, not distraction.)

Context Prioritization Hierarchy

- **CRITICAL** (always include) — Current code/content being modified, specific error messages, expected output format, active task requirements.
- **HELPFUL** (include if space permits) — Project structure, dependencies in use, coding conventions, related documentation.
- **OPTIONAL** (only if needed) — Historical decisions, future plans, tangential examples, background reference.

Production Patterns from Real Systems

The Static-to-Dynamic Spectrum

- **Static** — Always loaded (`CLAUDE.md`, `.cursorrules`). Predictable but stale; always costs tokens.
- **Tiered** — Hot/warm/cold layers loaded by role or trigger. Structured, requires infrastructure.
- **Dynamic** — Agent discovers and fetches on demand (Cursor, Claude Code grep/glob). Token-efficient, always fresh.
- **Self-improving** — Contexts evolve based on execution feedback (ACE framework). No manual maintenance.

Manus's 5 Production Strategies

- **Context compaction & summarization** — Reversible compaction (store file paths, not contents) plus lossy summarization triggered around 128K tokens.
- **Communication over shared context** — Fresh sub-agents for discrete tasks instead of sharing full conversation history.
- **Hierarchical action spaces** — Limit visible tools to ~20 atomic tools. Level 2: sandbox utilities. Level 3: code libraries.
- **Agent-as-tool pattern** — Sub-agents as deterministic functions with defined inputs, outputs, and JSON schemas.
- **Implementation best practices** — Avoid dynamic RAG-based tool retrieval (breaks KV cache). Set pre-rot thresholds at 70–80% utilization.

Static Context Files: The ETH Zurich Study

- **−3% success rate** — LLM-generated context files actually hurt performance.
- **+4% success rate** — Human-written context files give marginal improvement.
- **+20% cost** across both file types (14–22% more reasoning tokens).
- **Best practice** — Keep it minimal (10–20 lines), specific tooling only, don't duplicate README content, review quarterly.

Key Retrieval Approaches

- **Vector store (RAG)** — Classic chunking + embeddings + similarity search. Requires indexing and chunking decisions.
- **Agentic search** — An `llm.txt` file with URLs + descriptions; the agent fetches docs on demand. Extremely effective with good descriptions.
- **Context stuffing** — Feed all docs directly to the agent. Brute force; expensive and degrades quality.
- **Progressive disclosure** — Start small, expand as needed. File paths and links first; load full content only when needed.

The Compression Toolkit

- **Prompted summarization** — Compress with high recall. Exhaustive bullets so the agent knows whether to re-fetch the full context.
- **Auto-compaction** — Automatic compression nearing limits. Claude Code auto-compacts at 95% capacity.
- **Trimming** — Selective removal using heuristics. Keep only recent N messages, or LLM-scored relevance.
- **Prompt caching** — Avoid re-sending invariant context. System prompts and tool definitions: cache, don't reprocess.
- **Context offloading** — Write full results to disk, return a summary plus a pointer. The agent fetches on demand.

> **ORDER MATTERS**
> Summarization is lossy. Always offload to disk *before* summarizing, so the raw context survives as a safety net.

Course Demos Quick Reference

Every demo runs with `uv run` — no global install or virtualenv needed. Set `ANTHROPIC_API_KEY` in your environment, or `cp .env.example .env` at the repo root and add your key.

Session 1 — Introduction to Context Engineering

Run from `demos/live-ctx-intro-demo/code/`: `uv run 01_what_the_model_sees.py` … `04_agent_loop_growth.py`

- **01** what the model sees (one flat token list) · **02** context budget (needle-in-a-haystack) · **03** the four moves, minimal · **04** token growth in an agent loop, raw vs. trimmed.

Session 2 — Live-coded agent loops

- `uv run demos/live_simple_agent.py` — simplest agent: one calculator tool in a loop.
- `uv run agent.py` in `demos/live-agent-retrieval/` — sandboxed `find` / `sed` / `grep` tools; `…-plus-write-select-compress-isolate/` adds `write_file` (write only) + a chat, `uv run app.py`.

Session 2 — Agentic Document Retrieval (interactive TUI)

Run: `uv run app.py` in `demos/agentic-retrieval/`

- **Hand-rolled agent loop** — `self.messages` *is* the context window, made explicit and inspectable.
- **Progressive disclosure** — `list_documents` (cheap) → `search_documents` (moderate) → `get_document` (expensive).
- System prompt ~200 tokens (WRITE lever); tool definitions ~800 tokens of fixed overhead per call.
- TF-IDF search with scikit-learn — no vector DB needed for small knowledge bases.
- Slash commands: `/context` shows the live messages array, `/stats` shows token counts.
- *Teaching point:* every `append()` to messages is a permanent token cost.

Session 2 / 4 — Chat with Artifacts (web app)

Run: `uv run app.py` in `demos/chat-with-artifacts/` → open `http://127.0.0.1:8000`

- **Layered system prompt** — Layer 1 (fixed persona ~150 tok) + Layer 2 (static artifact schemas ~400 tok) + Layer 3 (dynamic state 0–200 tok).
- 7 artifact types: `selectable_options`, `flashcard_deck`, `checklist`, `flowchart`, `semantic_zoom`, `inline_quiz`, `concept_explorer`.

- **Structured output via `tool_choice`** — guaranteed JSON matching the schema. "The prompt provides the content; the schema provides the shape."
- **ArtifactRegistry** injects dynamic context: tracks what's been created, tells the model to build on existing artifacts.
- `structured_outputs_demo.py` — same prompt, 3 schemas, 3 output shapes, same input cost.

Session 3 — Diagnosing & Fixing Context Failures (taught live, no committed demo)

Poisoning, Distraction, Confusion, Clash — each run broken vs. fixed; see Failures to Fixes above.

Session 4 — Agent Skills: a deck from data

Run: `uv run demos/agents-with-skills/pptx_from_data_agent.py` (live-coded: `demos/live-agents-with-skills/ppt_agent.py`)

- Managed `pptx` skill + code execution: loaded only when needed; deck returned by `file_id` (`claude-opus-5-5`).

Bonus — Agentic RAG via the Agent SDK

Run: `uv run demos/full_agent_app.py` (optionally with a folder path)

- Custom MCP tools over the knowledge base, driven by `claude-agent-sdk`.
- Shows the same progressive-disclosure retrieval shape as Session 2, but wired through the SDK.

Bonus — Context Engineering Chat Agent Overview

In `demos/live-demo-chat-agent-ctx-eng-overview/`: `uv run chat.py` (from-scratch tool-use loop) · `uv run quiz_app.py` (open `http://127.0.0.1:8501`) · `uv run structured_output_example.py` (structured-output primer)

Session 5 — Tools & Techniques

- Claude Code primitives: `CLAUDE.md` as a routing table, custom slash commands, hooks, sub-agents, MCP.
- **Scheduled agents** — cron runs headless `claude -p`; see `demos/ctx-engineering-tools-claude-code/`.

Key Numbers to Remember

- **500K tokens** per run for naive deep-research agents · **$1–2** per single agent run.
- **~50–100 tool calls** in production agents · **83.9%** of context consumed by tool outputs.
- **n² attention complexity** — more tokens means exponentially more pairwise relationships to score.
- **95% capacity** — Claude Code auto-compaction trigger · **70–80%** — Manus pre-rot threshold.
- **46.9%** token reduction from Cursor's dynamic context discovery · **44%** performance improvement from dynamic tool loading.
- **~150–200 instructions** — frontier-LLM instruction-following limit; Claude Code's system prompt already uses ~50.

Core Principles

> **THE ONE-LINE VERSION**
> "Find the smallest set of high-signal tokens that maximize the likelihood of your desired outcome." — Anthropic, *Effective Context Engineering for AI Agents*

- **Context engineering is entropy reduction** — high-entropy intentions into low-entropy representations machines understand.
- **Systems, not prompts** — the full architecture of information flow: storage, management, usage.
- **Four operations: Write, Select, Compress, Isolate** — every strategy maps to one of these.
- **Quality over quantity** — bigger windows don't help; focused, relevant, well-managed context does.
- **Context hygiene is continuous** — monitor, prune, refresh, and validate throughout the lifecycle.
- **Agentic retrieval beats naive RAG** — simple tools (grep/glob) with progressive disclosure outperform complex pipelines.
- **Read tasks parallelize, write tasks don't** — sub-agents for research, single-threaded execution for code generation.
- **Offload before you summarize** — save raw context to disk first, then compress. Summarization is lossy.